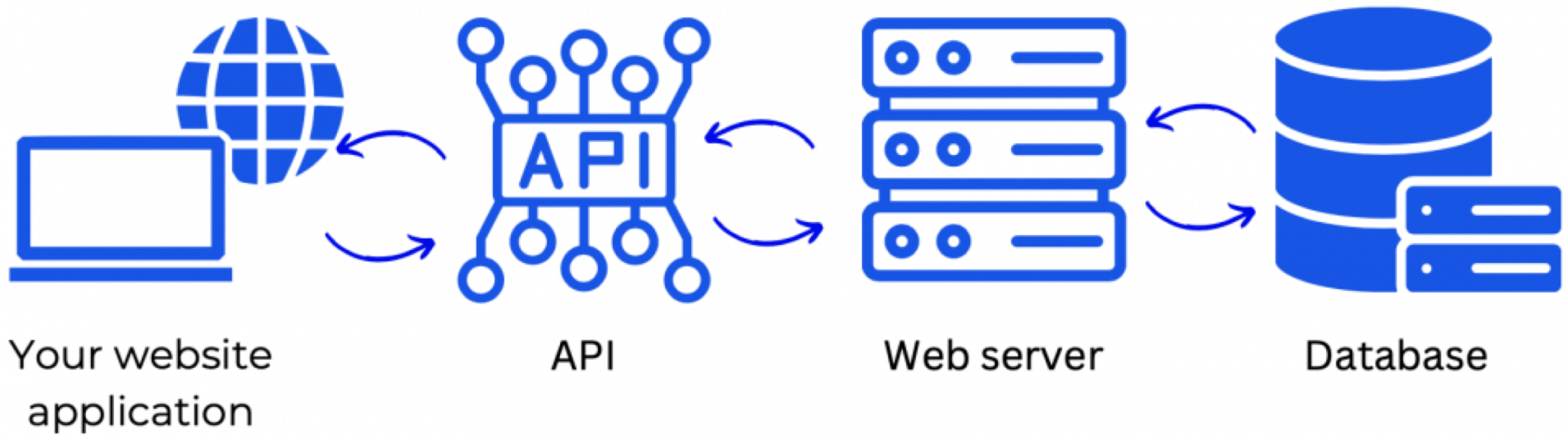


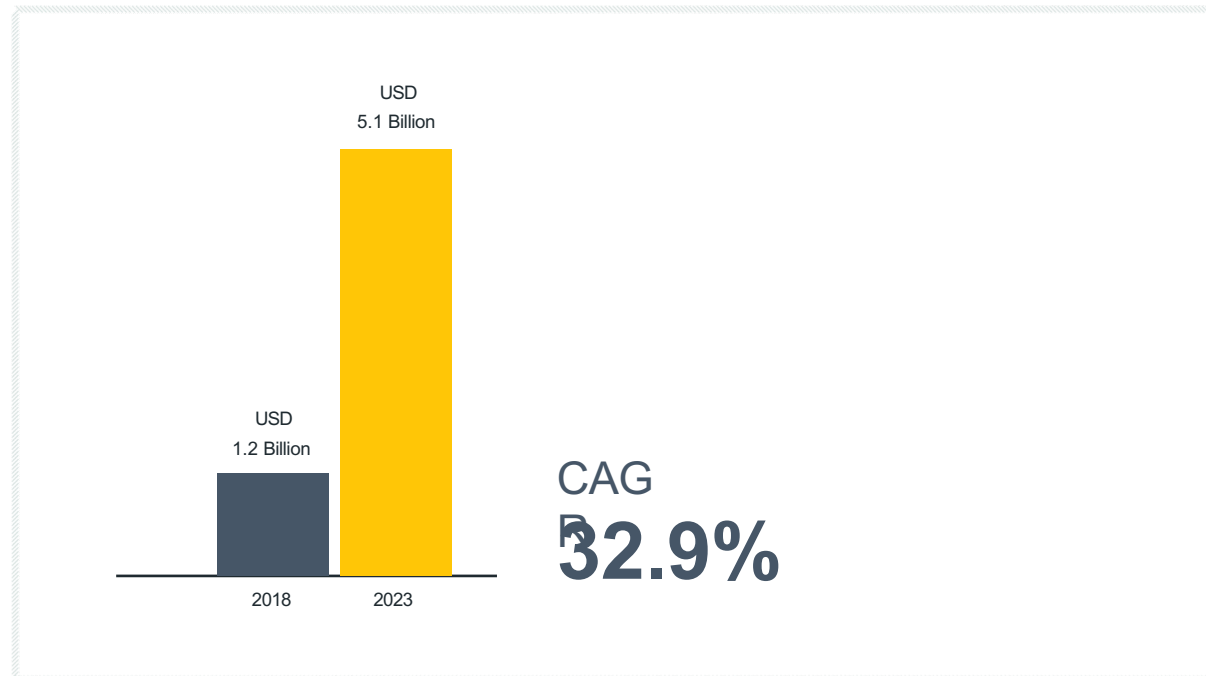


Secure Software Design and Development (CY-321)

API Security

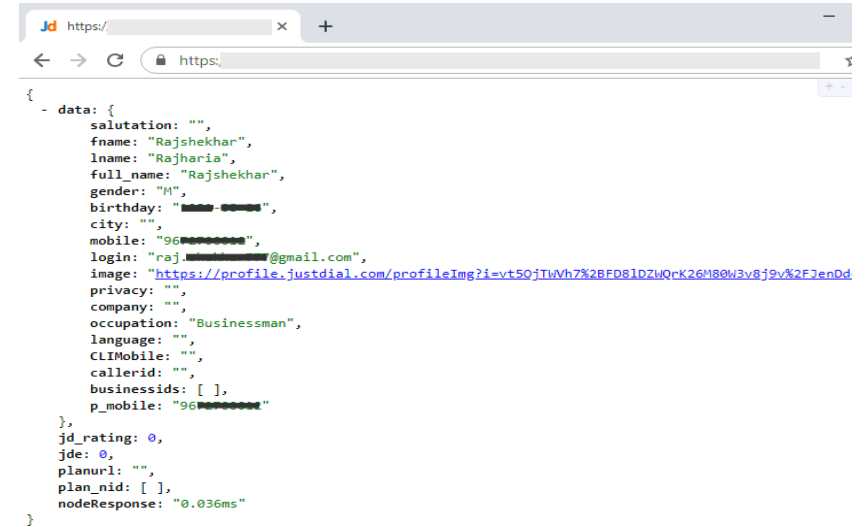


Reliance on API strategies is reflected in the explosion of tools to help manage and control access to those services. For instance, the global API management market is expected to grow at a compound annual growth rate (CAGR) of 32.9% to reach USD 5.1 billion according to [MarketsandMarkets](#).



Whats being Stolen?

▶ In 2019, a [data breach at JustDial](#) placed the data of more than 100 million users at risk due to an unprotected publicly accessible API endpoint.



```
{
  - data: {
    salutation: "",
    fname: "Rajshekhar",
    lname: "Rajharia",
    full_name: "Rajshekhar",
    gender: "M",
    birthday: "1990-01-01",
    city: "",
    mobile: "9600000000",
    login: "raj.00000000@gmail.com",
    image: "https://profile.justdial.com/profileImg?i=vt50jTtVh7%28FD81DZWQrK26M80W3v8j9v%2F2enDd",
    privacy: "",
    company: "",
    occupation: "Businessman",
    language: "",
    CLIMobile: "",
    callerid: "",
    businessids: [ ],
    p_mobile: "9600000000"
  },
  jd_rating: 0,
  jde: 0,
  planurl: "",
  plan_nid: [ ],
  nodeResponse: "0.036ms"
}
```

Whats being Stolen?

In 2018, an uncaught vulnerability in [Facebook's Developer API](#) exposed the personal data of nearly 50 million users.

Facebook says nearly 50m users compromised in huge security breach

Attack gave hackers ability to take over accounts in what is believed to be largest breach in Facebook's history



📷 Mark Zuckerberg: 'I think this underscores the attacks that our community and our services face.'
Photograph: Marcio Jose Sanchez/AP

Nearly 50m [Facebook](#) accounts were compromised by an attack that gave hackers the ability to take over users' accounts, Facebook revealed on Friday.

Whats being Stolen?



KrebsOnSecurity
In-depth security news and investigation

HOME ABOUT THE AUTHOR ADVERTISING/SPEAKING

Experian API Exposed Credit Scores of Most Americans

April 28, 2021

Big-three consumer credit bureau Experian just fixed a weakness with a partner website that let anyone look up the credit score of tens of millions of Americans just by supplying their name and mailing address, KrebsOnSecurity has learned. Experian says it has plugged the data leak, but the researcher who reported the finding says he fears the same weakness may be present at countless other lending websites that work with the credit bureau.



≡ **threatpost** Podcasts / Malware / Vulnerabilities / InfoSec Insiders

Dating Site Bumble Leaves Swipes Unsecured for 100M Users



WIRED BACKCHANNEL BUSINESS CULTURE GEAR IDEAS MORE ▾ SIGN IN SUBSCRIBE

DAN SALMON SECURITY JUN 26, 2019 9:00 AM

I Scraped Millions of Venmo Payments. Your Data Is at Risk

Opinion: Venmo makes sending and receiving money a social affair. But those emoji-laden payment descriptions leave you exposed to cyberattacks.



SECURITY Search 🔍 🛒 f t in y ≡

SIGN IN CREATE ACCOUNT

MANAGEMENT | CYBERSECURITY | SECURITY NEWSWIRE | CYBERSECURITY NEWS

Hacker behind LinkedIn scraping grabbed 700m profiles 'for fun'

A hacker created a database of information scraped from 700 million LinkedIn users after tricking the company's API and used the same technique to create a database on 533 million Facebook users.

API attack examples

Coinbase Unauthorized trading

USPS Account data harvesting

venmo Excess data exposure

Instagram Account takeover

bumble Account tampering

T-Mobile SEC reporting

OPTUS Ransom

experian 3rd party exposure

Examples highlight:

- Attacks are new and different from traditional threats
- Difficult to detect using conventional security tools
- API attacks bypass perimeter defenses undetected
- Developers must be involved in security from the start

Whats being Stolen?

A majority of respondents to a 2021 **Salt Security survey** said they had delayed the launch of an application due to **API security concerns**. Of those same respondents, 91 percent also reported having uncovered an **API vulnerability** in the previous year, indicating their security concerns are also born from experience

API Storm

API Explosion

83%

of all Internet
traffic is
API traffic



APIs Under Attack

#1

“API attacks will
become the
**most frequent
attack vector**”

Gartner

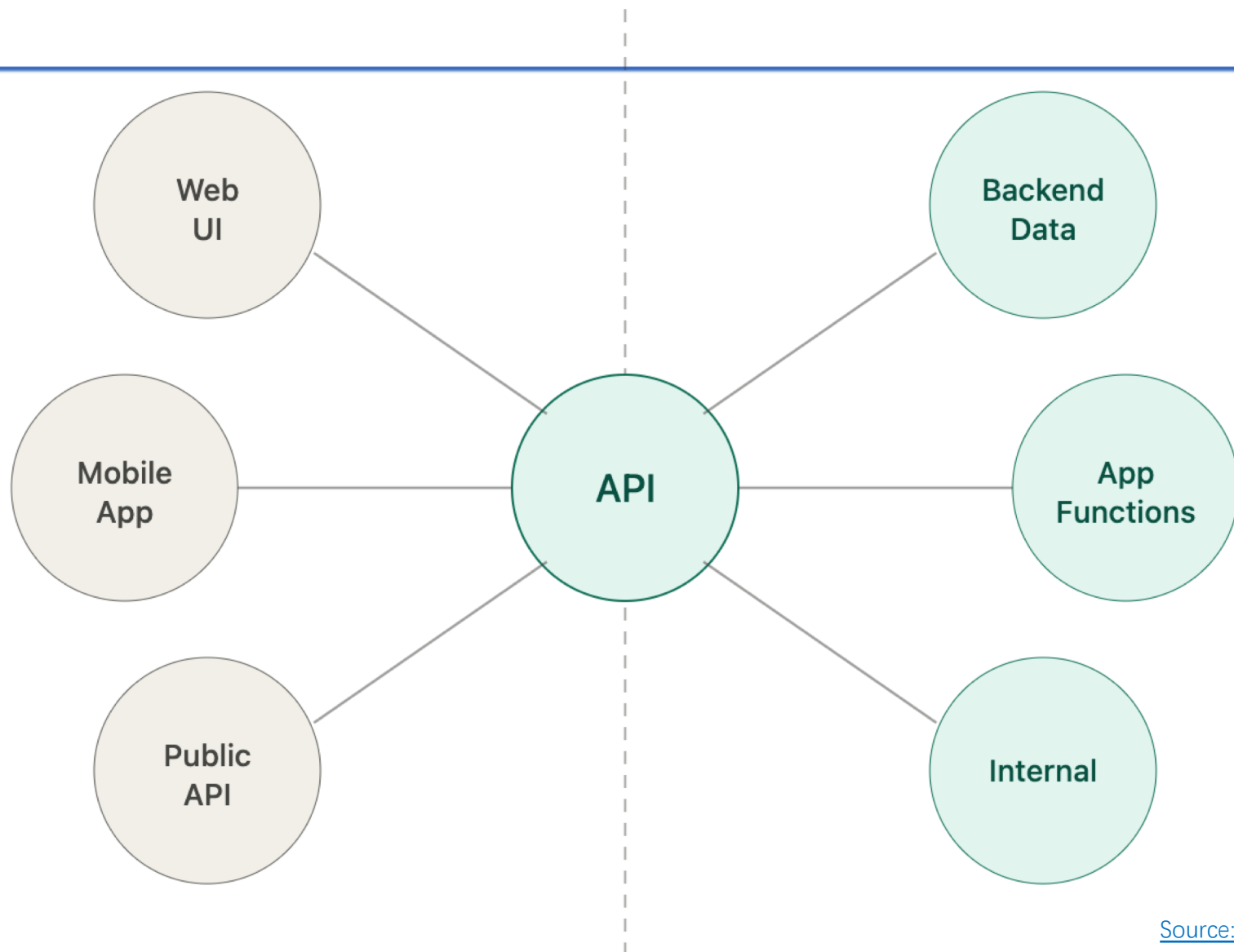
APIs Under-Secured

4%

of testing is Security

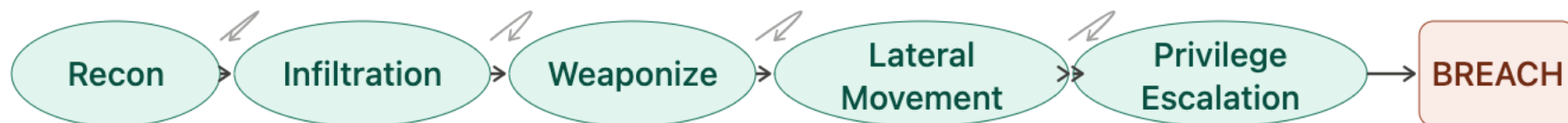


[Source: APIsec University](#)



Source: [APIsec University](#)

Normal Attack



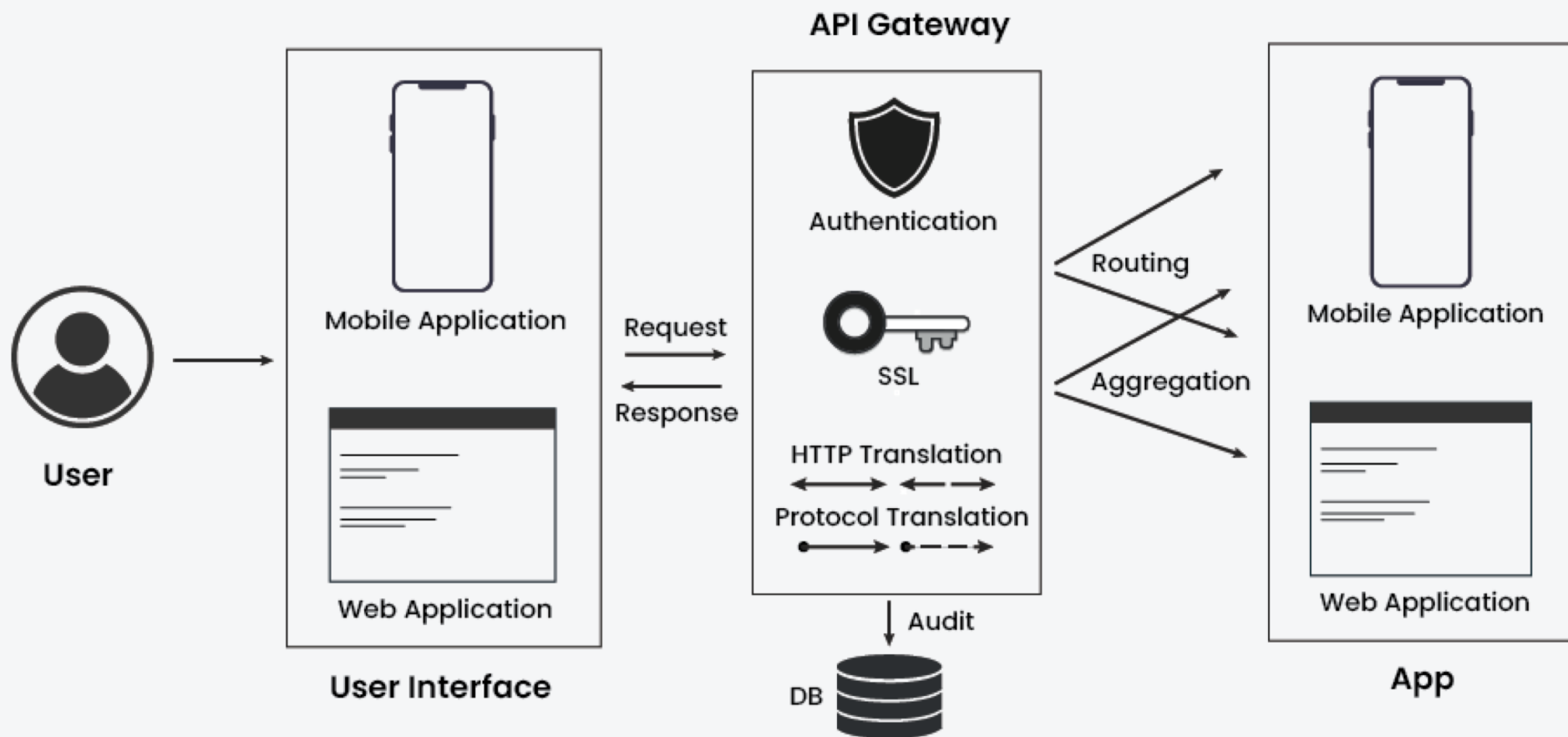
API Attack



Source: [APIsec University](#)

- When talking about **API security**, first we must understand where it should be addressed.
- Typically, an API is created by a set of API developers to perform a set of **business operations** using specified **business logic**.
- If all the **security aspects** are also included within the API itself, the **implementation logic** of the API becomes more **complex**

- In addition, a particular API can be accessed by a wide range of devices and applications (in most cases, these applications require different **security mechanisms**).
- Therefore, it is important to comprehensively address **API security** concerns and prevent misuse.
- To address these requirements, the best practice is to offload **API security** responsibilities to an **API gateway**.



OWASP API Security Top 10

API1 Broken Object Level
Authorization

API2 Broken Authentication

API3 Broken Object Property
Level Authorization

API4 Unrestricted Resource Consumption

API5 Broken Function Level Auth

API6 Unrestricted Access to
Sensitive Business Flows

API7 Server Side Request Forgery

API8 Security Misconfiguration

API9 Improper Inventory Management

API10 Unsafe Consumption of APIs

Broken Object Level Authorization

What is it?

- Broken Authorization refers to flaws in logic/rules governing access
- Most common and damaging API vulnerability
- Very difficult to detect in runtime
- Critical to test for BOLA in pre-production

Examples, Risk Exposure

- Significant risk of data loss
- Example: can User A access User B data?
- Fraudulent transactions

```
▼ Request Payload view source  
▼ {client_order_id: "274fce73-edd3-4fc5-b2a3-86290cd70698", product_id: "ETH-EUR", side: "SELL",...}  
  client_order_id: "274fce73-edd3-4fc5-b2a3-86290cd70698"  
  ▼ order_configuration: {limitLimitGtc: {baseSize: "0.02433012", limitPrice: "3000", postOnly: false}}  
    ▼ limitLimitGtc: {baseSize: "0.02433012", limitPrice: "3000", postOnly: false}  
      baseSize: "0.02433012"  
      limitPrice: "3000"  
      postOnly: false  
      product_id: "ETH-EUR"  
      side: "SELL"  
      source_account_id: "74f5810e-bda4-5277-ba28-90cb98798984"  
      target_account_id: "e64ba5fc-7db3-5e04-81ee-cedfd4fb2543"
```

Trade Confirmation!

coinbase

Sign up



Retrospective: Recent Coinbase Bug Bounty Award

By Author

Company, February 18, 2022, 3 min read time



At Coinbase, our number one priority is ensuring that we uphold our security commitments to our customers.

On February 11, 2022, we received a report from a third-party researcher indicating that they had uncovered a flaw in Coinbase's trading interface. We

Root Cause

The underlying cause of the bug was a missing logic validation check in a Retail Brokerage API endpoint, which allowed a user to submit trades to a specific order book using a mismatched source account. This API is only utilized by our Retail Advanced Trading platform, which is currently in limited beta release.



Anyone here can get me a direct line with someone at @coinbase , preferably management or dev team, possibly @brian_armstrong himself?

I'm submitting a hacker1 report but I'm afraid this can't wait. Can't say more either, this is potentially market-nuking.

DMs open.

10:16 AM · Feb 11, 2022

Featured Article

Peloton's leaky API let anyone grab riders' private account data

But the company won't say if it has evidence of malicious exploitation

Zack Whittaker @zackwhittaker / 4:00 AM PDT • May 5, 2021

 Comment

Broken Object Property Level Authorization

What is it?

- Exploit of endpoints by reading and/or modifying values of objects
- Ability to update object elements (“**mass assignment**”)
- Revealing unnecessary sensitive data (“**excessive data exposure**”)

Examples, Risk Exposure

- User is able to set “account-type=premium”
- API returns excessive, unnecessary details
- Exposing sensitive user data
- Data theft

Broken Object Property Level Authorization

≡ WIRED

SUBSCRIBE

DAN SALMON

SECURITY JUN 26, 2019 9:00 AM

I Scraped Millions of Venmo Payments. Your Data Is at Risk

After proxying my phone traffic through my laptop, I watched the network traffic as I navigated through the app. I noticed that when you open the Venmo home page, you're shown a live feed of transactions being made by strangers. I could see a public API endpoint that was returning the data for this feed, meaning that anyone could make a GET request (like a simple page load) to see the latest 20 transactions made on the app by anyone around the world. To my surprise, this endpoint was accessible even outside the app, with no authorization needed. After some experimenting, I found that I could make two requests for transaction data per minute, per IP address.

ZDNET



Home / Tech / Security

Venmo has no good reason to make user transactions public by default

"I used Venmo's public API to pull in all public transactions of 2017 -- a total of 207,984,218 transactions," he said in an email. "By looking through them, I learned a scary amount about Venmo users. I was able to follow a drug dealer's sales, watch a couple fight viciously on Valentine's Day, and learn exactly how many mangos a Santa Barbara, CA food cart sells each week."

Broken Object Property Level Authorization

```
let user = {  
  id: 1,  
  username: "Ben",  
  role: "user",  
  accountType: "basic"  
},
```

```
app.put("/update-profile", (req, res) => {  
  Object.assign(user, req.body);  
  
  res.json({  
    message: "Profile updated",  
    user: user  
  });  
});
```

Broken Object Property Level Authorization

```
app.put("/update-profile", (req, res) => {  
  const allowedFields = ["username"];  
  
  allowedFields.forEach(field => {  
    if (req.body[field]) {  
      user[field] = req.body[field];  
    }  
  });  
  
  res.json({  
    message: "Profile updated safely",  
    user: user  
  });  
});
```

Unrestricted Resource Consumption

What is it?

- Formerly “**Lack of Resources and Rate Limiting**”
- Abuse of APIs due to high volumes of API calls, large requests, etc.

Examples, Risk Exposure

- Missing/inadequate rate controls
- Mass data harvesting
- Execution timeouts
- Max memory, number of files, upload size
- Excessive operations in single request
- Excessive records returned in a request
- Denial of service
- Performance impact

Rate Limiting

Controls the number of API requests a user or system can make in a given time.

If not implemented correctly, attackers can **flood** the API with excessive requests, causing service disruption or account takeover attempts (brute force attacks)

Rate Limiting (Insecure)

```
from flask import Flask, request

app = Flask(__name__)

@app.route("/login", methods=["POST"])
def login():
    username = request.form["username"]
    password = request.form["password"]

    if username == "admin" and password ==
"password123":
        return {"message": "Login successful"}
    return {"message": "Invalid credentials"},
401
```

Rate Limiting (Secure)

```
from flask import Flask, request
from flask_limiter import Limiter
from flask_limiter.util import get_remote_address
```

```
app = Flask(__name__)
limiter = Limiter(get_remote_address, app=app,
                  default_limits=["5 per minute"])
```

```
@app.route("/login", methods=["POST"])
@limiter.limit("5 per minute")
```

```
def login():
```

```
    username = request.form["username"]
```

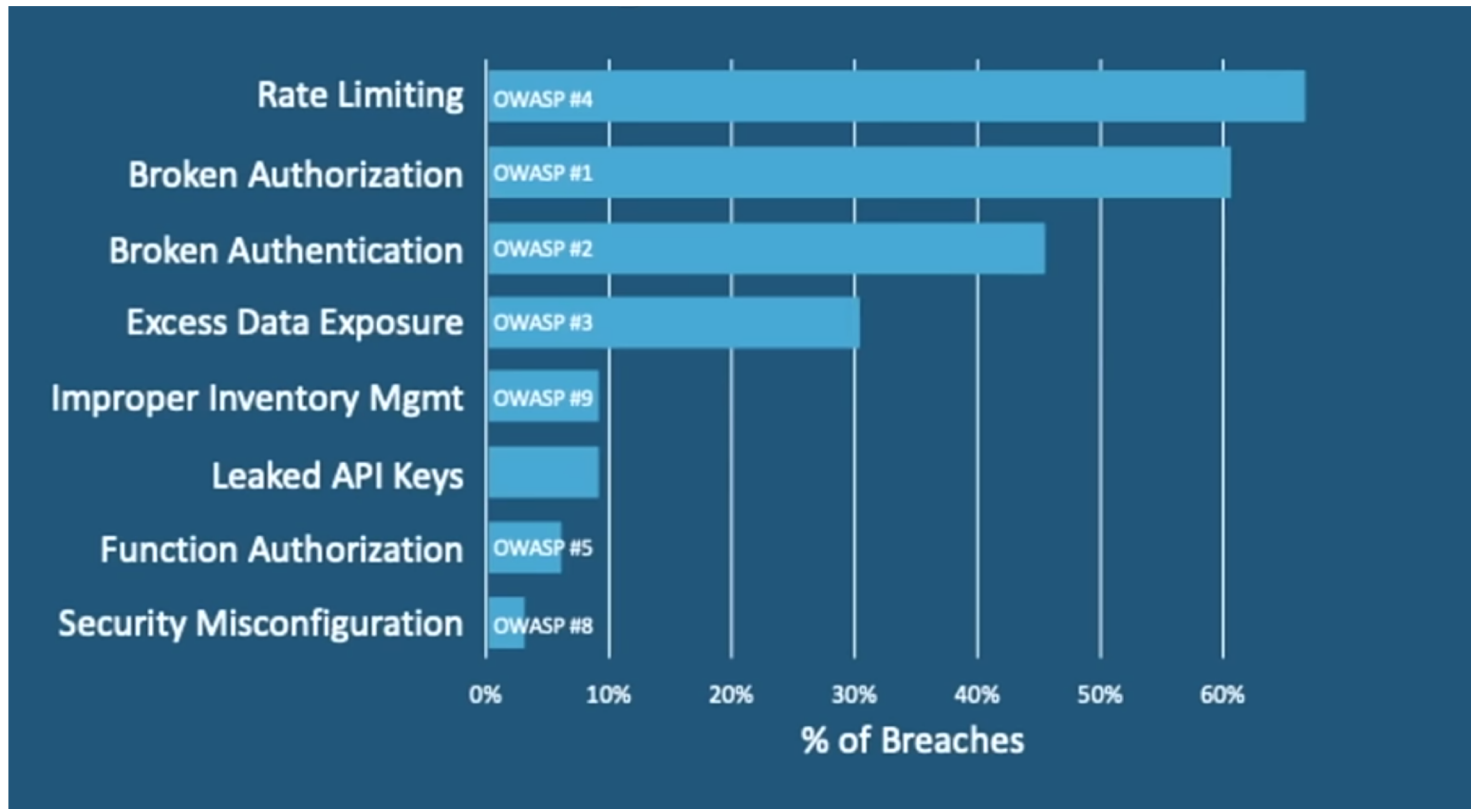
```
    password = request.form["password"]
```

```
    if username == "admin" and password == "password123":
```

```
        return {"message": "Login successful"}
```

```
    return {"message": "Invalid credentials"}, 401
```

API Breach Analysis



80%

Why API security?

- Its the right thing to do
- Reduce risk and harm
- We have to - compliance

How important is it?

- Is API security a priority?
- Is the business investing in it?
- Why now?

Threat Modeling

Identify: APIs, business flows, data, access paths

Assess: vulnerabilities, logic flaws, access controls, 3rd party risk

Probability: examine the likelihood of an attack

Impact: understand the damage, loss, consequences of an attack

Mitigation: develop a plan to address the risk

What do you have that attackers want?

- **Personal information:** consumer data
- **Financial information:** banking data, credit cards
- **Corporate data:** IP, corporate data
- **Fraud:** financial profit, free use
- **Critical infrastructure:** activism, mayhem, terrorism

How are APIs used in your business?

- Web site functionality
- Mobile application
- Customer/partner API access
- Internal microservices
- 3rd party data and services

Three Pillars of API Security

Governance

Developing secure APIs

Monitoring

Detecting threats in
production

Testing

Ensuring APIs are free
of flaws

JWT (JSON Web Token) is a compact, URL-safe token format used for authentication and data exchange. It consists of three parts:

1.Header: Contains metadata, such as the signing algorithm (e.g., HS256, RS256).

2.Payload: Contains claims (user data, roles, expiration, etc.).

3.Signature: Ensures integrity by signing the header and payload with a secret key.

Instead of requiring users to share their passwords with third-party apps, OAuth 2.0 lets them grant access via **tokens**. Here's a simplified process:

User Initiates Authorization → A user wants to log in to a third-party app (e.g., a fitness tracker) using their Google account.

App Requests Authorization → The fitness app redirects the user to Google's OAuth authorization server.

User Grants Permission → The user logs in and grants the fitness app access to specific data (e.g., Google Fit).

Authorization Server Issues a Token → Google generates an **access token** and sends it back to the fitness app.

App Uses the Token → The fitness app can now access Google Fit API securely without needing the user password.

Broken Authorization

APIs should **restrict access** based on user roles and permissions.

If broken authorization exists, unauthorized users can access **data or functions they shouldn't be allowed to**.

Uber API had a vulnerability where a normal user could **change their account type to an admin** by modifying the API request.

Broken Authorization (Insecure)

```
@app.route("/account/<account_id>", methods=["GET"])
def get_account(account_id):
    user = get_logged_in_user() # Fetch the authenticated user
    account = Account.query.filter_by(id=account_id).first()

    if not account:
        return jsonify({"error": "Account not found"}), 404

    return jsonify({"account_id": account.id, "balance":
account.balance})
```

Broken Authorization (Insecure)

The API assumes the user requesting the account data has access to it but does not check if account_id belongs to user. An attacker could simply change the account_id in the request to access someone else's bank details.

```
@app.route("/account/<account_id>", methods=["GET"])
def get_account(account_id):
    user = get_logged_in_user() # Fetch the authenticated user
    account = Account.query.filter_by(id=account_id).first()

    if not account:
        return jsonify({"error": "Account not found"}), 404

    return jsonify({"account_id": account.id, "balance":
account.balance})
```

Broken Authorization (Secure)

```
@app.route("/account/<account_id>",
methods=["GET"])
def get_account(account_id):
    user = get_logged_in_user()
    account =
Account.query.filter_by(id=account_id,
user_id=user.id).first() # Authorization check!

    if not account:
        return jsonify({"error": "Unauthorized
access"}), 403

    return jsonify({"account_id": account.id,
"balance": account.balance})
```

Broken Authentication

Broken authentication happens when an API **fails to properly verify user identity**, allowing attackers to **impersonate legitimate users**

Facebook exposed **access tokens** of **50 million users** due to a flaw in its API authentication.

Broken Authentication

```
@app.route("/dashboard", methods=["GET"])
def dashboard():
    token = request.headers.get("Authorization")

    if not token:
        return jsonify({"error": "Unauthorized"}), 401

    try:
        decoded = jwt.decode(token, SECRET_KEY,
    algorithms=["HS256"]) # Decoding the JWT
    except jwt.ExpiredSignatureError:
        return jsonify({"error": "Token expired"}), 401
    except jwt.InvalidTokenError:
        return jsonify({"error": "Invalid token"}), 401

    return jsonify({"message": "Welcome to your dashboard"})
```

Broken Authentication

Some developers make a critical mistake by allowing `algorithms=["none"]`, which means an attacker can create an unsigned token (`alg: none`) and bypass authentication.

```
decoded = jwt.decode(token, SECRET_KEY, algorithms=["none"]) # Vulnerable code
```

API Breach Analysis

Excess Data Exposure

APIs should only return **necessary data**. If they return **too much information**, attackers can extract sensitive details from responses.

API Breach Analysis

Excess Data Exposure (Insecure)

```
@app.route("/user/<user_id>", methods=["GET"])
def get_user(user_id):
    user =
    User.query.filter_by(id=user_id).first()

    if not user:
        return jsonify({"error": "User not
found"}), 404

    # Exposing too much information!
    return jsonify(user.__dict__)
```

API Breach Analysis

Excess Data Exposure (**Insecure**)

If the User model contains sensitive fields (e.g., password, security questions), this API will expose them in the response.

```
{  
  "id": 1,  
  "username": "john_doe",  
  "email": "john@example.com",  
  "password": "hashed_password_here",  
  "security_question": "Your first pet?"  
}
```

API Breach Analysis

Excess Data Exposure (Secure)

```
@app.route("/user/<user_id>", methods=["GET"])
def get_user(user_id):
    user = User.query.filter_by(id=user_id).first()

    if not user:
        return jsonify({"error": "User not found"}), 404

    return jsonify({
        "id": user.id,
        "username": user.username,
        "email": user.email # No sensitive data!
    })
```

Avoiding API Abuse

Use Role-Based or Attribute-Based Access Control (RBAC/ABAC)

Define user roles (admin, user, editor) and limit access to API endpoints based on role.

Enforce User-Specific Data Access

Ensure that users can only access their own data

Avoiding API Abuse

Use OAuth2 with Scopes

Implement OAuth2 and define fine-grained permissions.

Use API Gateway or Data Filtering Middleware

Centralize response filtering to **remove unnecessary fields.**

Questions??

zubair.ahmad@giki.edu.pk

Office: G14 FCSE lobby